SENIOR PROJECT PROPOSAL · KMUTT

Co-optimization of KV-Cache Management

High-throughput, long-context LLM inference on consumer GPUs



Computer Engineering · Academic Year 2026–2027

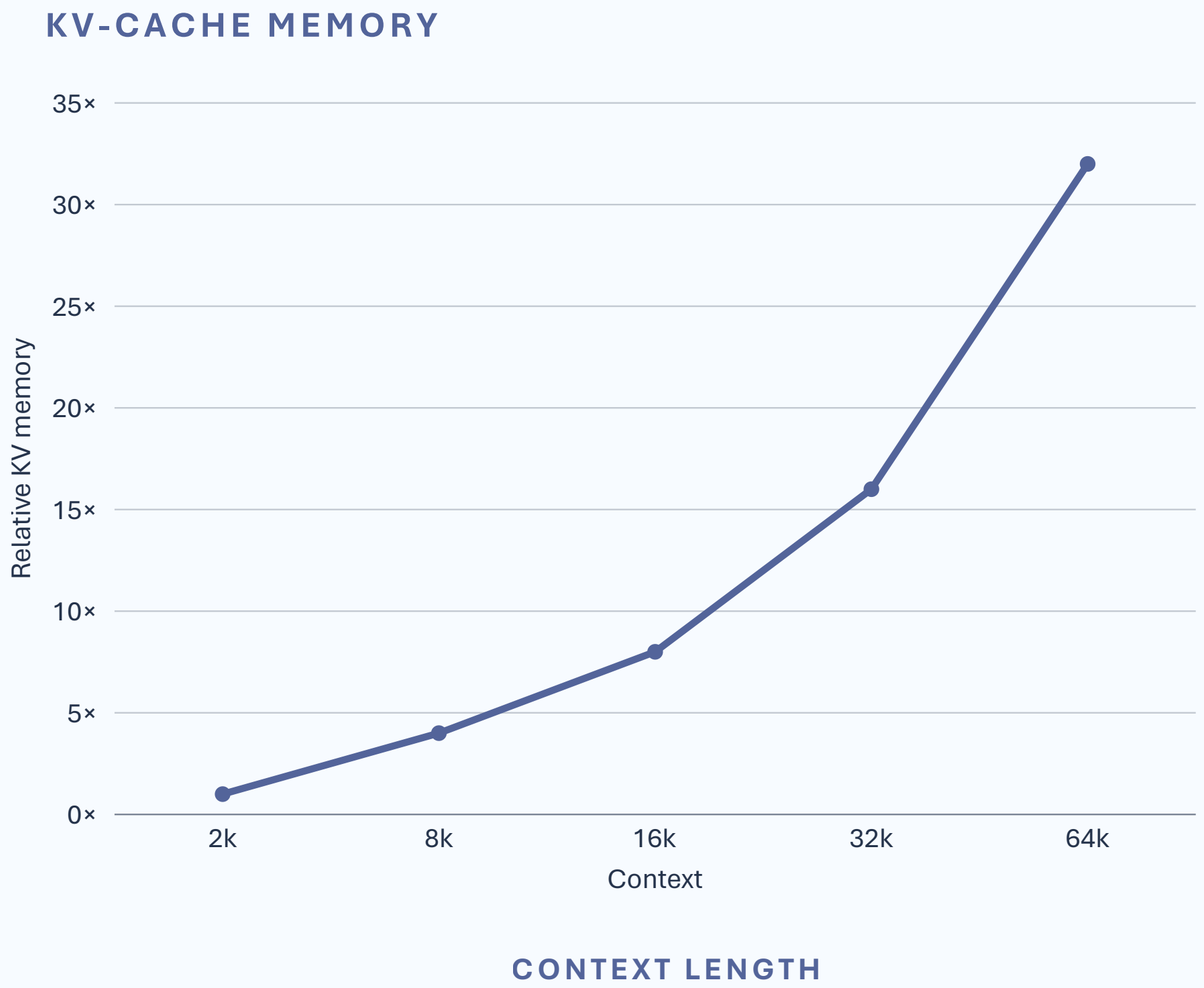
Advisors: Dr. Aye Hninn Khine
Co-Advisors: Dr. Boonyarit Changaival
By:
1. Kantawich Limwilai Student ID: 66070503406
2. Khem Ingkapat Student ID: 66070503408
3. Sirawat Sanboonsong Student ID: 66070503456
4. Run Prajunban Student ID: 66070503480

The KV-cache bottleneck

With Auto-regressive model, every generated token preserves keys and values for every layer. Context grows; memory follows linearly.

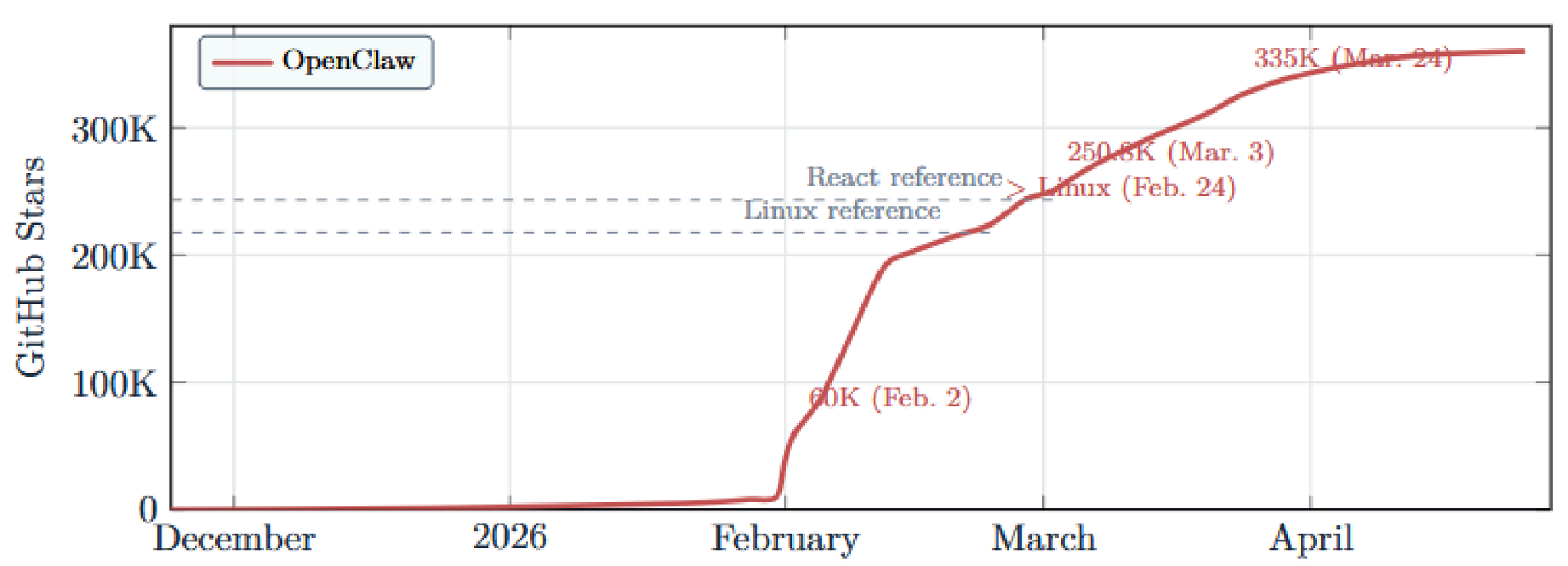
Memory ∝ context length

On a consumer GPU, the cache—not compute—can become the hard ceiling for long-context decoding.



S. Verma and N. Vaidya, "Mastering LLM Techniques: Inference Optimization," NVIDIA Technical Blog, 2023.

Local LLM is Free(?)



Source: Li et al. (2026), Fig. 3

Rise of Local LLM + Harness, but at what cost ?

Three objectives, one constraint envelope

Enable long-context, multi-turn LLM inference on 8-16 GB consumer GPUs by jointly managing KV-cache compression, placement, and transfer scheduling in one policy layer.

1 · Fit the Concurrent Context

2 · Acceptable Delay

3 · Acceptable Accuracy

Deliver an adaptive runtime that jointly chooses what to keep, how precisely to store it, and when to move it.

Three dominant paradigms

Keep the valuable tokens

Attention sinks, heavy hitters, and recency policies reduce active sequence length.

Risk · irreversible loss

QUANTIZATION

Use fewer bits per token

Low-bit KV representations trade numerical fidelity for a larger effective cache.

Risk · error accumulation

OFFLOADING

Move cold states to host

CPU memory extends capacity while prefetch tries to conceal a slower interconnect.

Risk · latency bubbles

1. **Verify on Wiki-text and PG-19(Book) Datasets (QServe, InfiniGen, StreamingLLM, and H2O) with Perplexity**
2. **Normally research perform on server-grade GPU (eg. A100, etc.)**

The missing layer is orchestration

	H2O	StreamingLLM	QServe	FlexGen	InfiniGen	ShadowKV	ArkVale
Spatial				✓	✓	✓	✓
Temporal					✓		✓
Structural	✓	✓	✓	✓		✓	

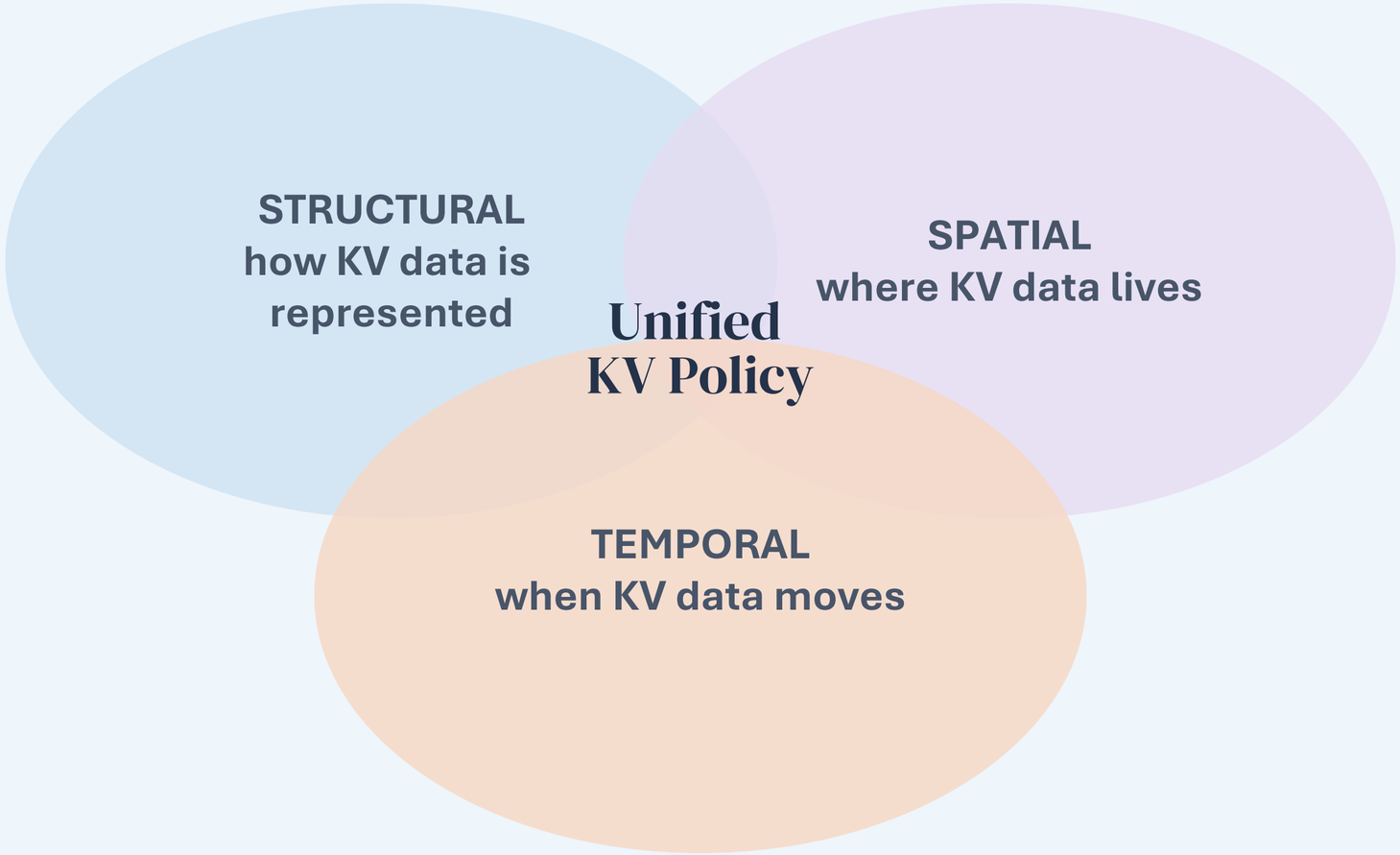
Existing systems usually optimize one axis, sometimes two—rarely all three under consumer-memory and PCIe constraints.

CO-DESIGN OPPORTUNITY

Research hypothesis · Coordinated decisions can outperform the sum of isolated techniques.

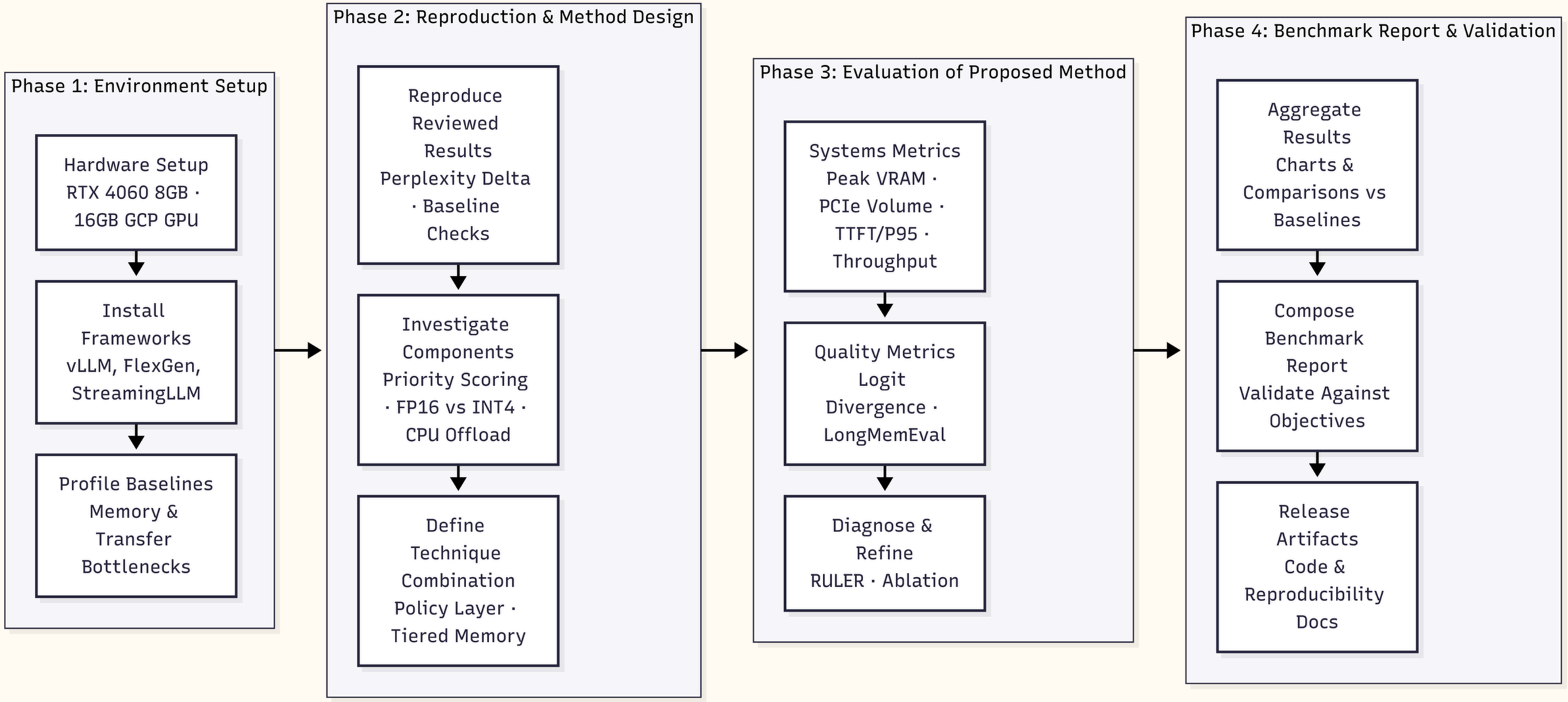
Sources: H2O (Zhang et al., NeurIPS 2023); StreamingLLM (Xiao et al., ICLR 2024); QServe (Lin et al., MLSys 2025); FlexGen (Sheng et al., ICML 2023); InfiniGen (Lee et al., OSDI 2024); ShadowKV (Sun et al., 2024); ArkVale (Chen et al., NeurIPS 2024).

A 3-axis co-optimization framework



Jointly optimize quality, memory footprint, and decode latency at runtime.

High-level Research & Development Process

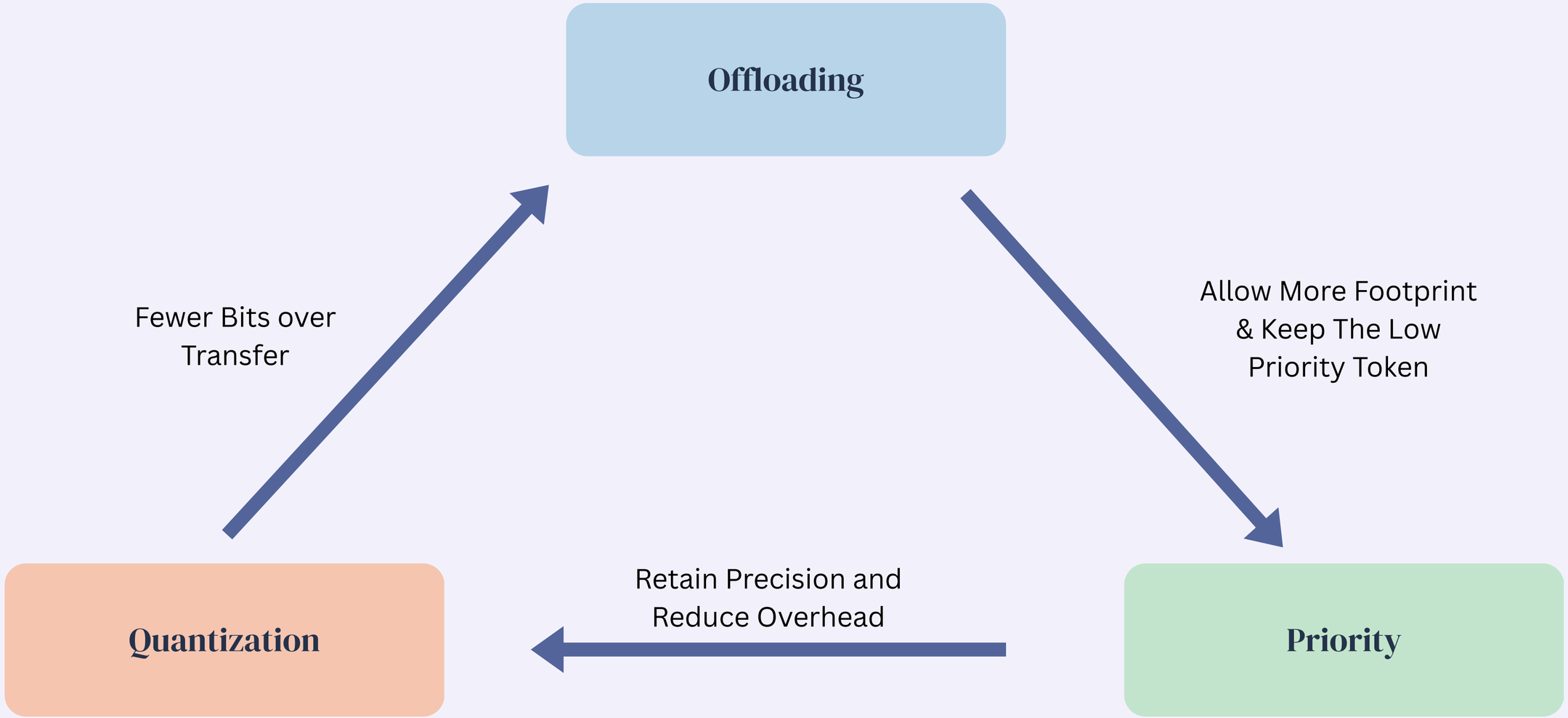


Candidate Components

Component	Axis	Borrowed from	What we adapt
Attention-based priority scoring	Structural	H2O (accumulated attention), StreamingLLM (sinks plus recent window)	Score per token, and *demote* instead of permanently evict
Selective quantization	Structural	FlexGen , QServe	FP16 for high-priority tokens, 4-bit for the rest
Tiered placement with CPU backup	Spatial	ArkVale (recallable backup), ShadowKV (partial offload)	Compressed low-priority tokens in host RAM, within a fixed VRAM budget
Prefetch and async transfer	Temporal	InfiniGen (prefetch), FlexGen	Overlap PCIe and CPU work with GPU decode (stream pipelining)

Sources: H2O (Zhang et al., NeurIPS 2023); StreamingLLM (Xiao et al., ICLR 2024); QServe (Lin et al., MLSys 2025); FlexGen (Sheng et al., ICML 2023); InfiniGen (Lee et al., OSDI 2024); ShadowKV (Sun et al., 2024); ArkVale (Chen et al., NeurIPS 2024).

Component Interactions



A testbed we can actually reproduce

HARDWARE

RTX 4060 8 GB (Local) · Up to 16 GB (Cloud / CPE Cluster)

Simulates 10–15 Concurrent SME Users

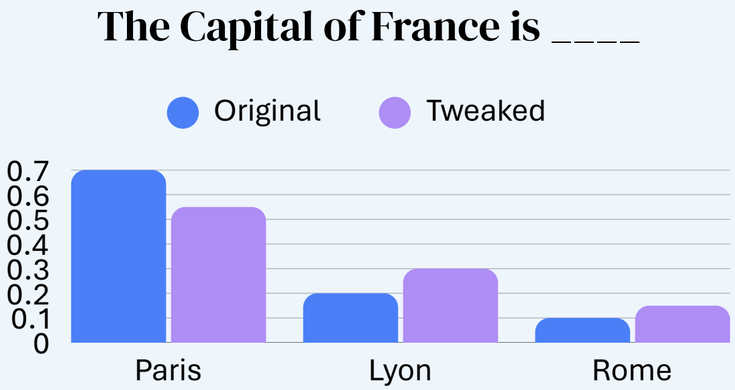


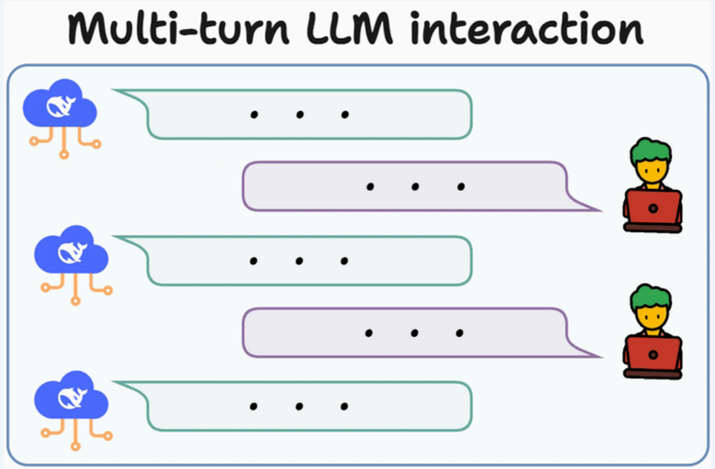
MODEL SUITE

Llama-3, Qwen-3, Gemma 3, Mistral

Axis	Metric	What it tells us
Spatial	Peak VRAM	Maximum GPU memory used, and the longest context that runs without OOM
Spatial	Host RAM delta	Host memory at launch vs. peak while holding offloaded cache (16 GB baseline). The cache moves to RAM, so this completes the memory picture.
Temporal	PCIe transfer volume	Bytes moved between host and GPU per token or turn, and % reduction vs. baselines (~75% expected)
Temporal	TTFT	Time to first token, cold start and warm start (prefix reuse), plus P95 under 10-15 simulated users
Temporal	Throughput and inter-token latency	Tokens/s and ms between tokens; confirms offloading doesn't stall generation. P95 reported if stalls appear.

Quality Metrics





Logit Divergence

How far the compressed model's next-token predictions drift from the full-precision model on the same input. It catches errors from eviction, offload, transfer, and dequantization at the output, without inspecting layers.

LongMemEval

Memory in long, multi-turn assistant dialogue, using a subset covering its five abilities: recalling facts, reasoning across sessions, updating outdated information, resolving relative time, and declining unanswerable questions

Contribution, So Far

Kantawich Limwilai
66070503406

Baseline Workload Research and Objective Definition

+ Quality Evaluate & Data Presentation

Khem Ingkapat
66070503408

State-of-the-Art Method and Quality Metrics Review

+ Core System Builder and Validator

Sirawat Sanboonsong
66070503456

Hardware Metrics and Testbed Define and Reseach

+ Infrastructure, pipeline, and testing

Run Prajunban
66070503480

State-of-the-Art Method Review and Manage Data Repository

+ Analysis, profiling, and tuning

First Semester Gantt Chart (Aug - Dec 2026)

Member | TEAM / All | KHEM | RUN | BLUEBIRD | NEW

Work Breakdown Structure / Tasks	Assignee	August		September				October				November				December			
		w3	w4	w1	w2	w3	w4	w1	w2	w3	w4	w1	w2	w3	w4	w1	w2	w3	w4
1. Literature Review & Workload Definition																			
Study SOTA KV-cache compression, eviction, and offloading mechanisms	KHEM & RUN																		
Analyze system requirements, profile workloads, and select benchmark datasets	RUN																		
2. Environment Setup & Baseline Replication																			
Build testbed on target consumer GPU (RTX 4060 8GB/16GB) & install frameworks	NEW																		
Replicate and profile baseline systems (vLLM, FlexGen, StreamingLLM)	KHEM & RUN																		
Perform system-level bottleneck profiling (Compute, Memory, and I/O constraints)	NEW & RUN																		
3. Architecture Co-Design & System Implementation																			
Design 3-axis strategy (Priority Engine, Selective Quantization, CPU Offloading)	TEAM																		
Implement memory tiering hierarchy and dynamic KV Cache Manager	KHEM																		
Integrate asynchronous transfer pipeline with kernel/stream overlap pipelining	NEW & RUN																		
4. Milestones & Reporting																			
Complete CITI Program Ethics Exam & Proposal Documentation [Milestone]	TEAM																		
Deliver the onsite proposal defense [Milestone]	TEAM																		
Refine prototype for validation & draft Semester 1 Progress Report	BLUEBIRD & NEW																		
Deliver the onsite progress presentation & defense [Milestone]	TEAM																		

Second Semester Gantt Chart (Jan - May 2027)

Member		TEAM / All	KHEM	RUN	BLUEBIRD	NEW

Work Breakdown Structure / Tasks	Assignee	January			February				March				April				May	
		w2	w3	w4	w1	w2	w3	w4	w1	w2	w3	w4	w1	w2	w3	w4	w1	w2
1. System Validation & Output Correctness																		
Execute numerical verification (KL divergence & Perplexity Delta)	KHEM																	
Conduct initial end-to-end system tuning & memory defragmentation checks	RUN & KHEM																	
2. Comprehensive Performance Benchmarking																		
Benchmark macro latency metrics (TTFT, TPOT, and tail latency)	RUN																	
Conduct stress testing across concurrency (1-8 sessions) & context (up to 32K)	NEW																	
Evaluate positional retrieval (RULER) & ong-term memory adherence (LongMemEval)	BLUEBIRD																	
3. Ablation Studies & Kernel Optimization																		
Isolate performance gains (priority scoring vs. offloading vs. compression)	KHEM																	
Analyze system trade-offs (CPU delegation vs. PCIe transfer bottlenecks)	RUN & NEW																	
Apply final bottleneck tuning & kernel optimizations based on test data	KHEM & RUN																	
4. Final Thesis & Artifact Preparation																		
Aggregate experimental data & generate publication-grade chart visualizations	BLUEBIRD																	
Draft the comprehensive technical thesis	TEAM																	
Submit open-source benchmarking artifacts & reproducibility documentation	NEW																	
Prepare slide deck, conduct rehearsals, and deliver Senior Project Defense [Milestone]	TEAM																	

References & open discussion

SELECTED REFERENCES

- H. Sun et al., "ShadowKV: KV Cache in Shadows for High-Throughput Long-Context LLM Inference," 2024.
- R. Chen et al., "ArkVale: Efficient Generative LLM Inference with Recallable Key-Value Eviction," NeurIPS 2024.
- G. Xiao et al., "Efficient Streaming Language Models with Attention Sinks," ICLR 2024.
- W. Lee et al., "InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management," OSDI 2024.
- Z. Zhang et al., "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," NeurIPS 2023.
- Y. Sheng et al., "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," ICML 2023.
- Y. Lin et al., "QServe: W4A8KV4 Quantization and System Co-Design for Efficient LLM Serving," MLSys 2025.
- J. L. Ruiz Williams, "HeadQ: Model-Visible Distortion and a Prototype KV Quantizer-Corrector Codec," 2026.
- W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," SOSP 2023.
- J. Jiang et al., "Towards Efficient Large Language Model Serving: A Survey on System-Aware KV Cache Optimization," Findings of ACL, 2026.
- S. Verma and N. Vaidya, "Mastering LLM Techniques: Inference Optimization," NVIDIA Technical Blog, 2023.
- D. Wu et al., "LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory," ICLR 2025.
- C.-P. Hsieh et al., "RULER: What's the Real Context Size of Your Long-Context Language Models?" 2024.
- D. Li et al., "How Long Can Open-Source LLMs Truly Promise on Context Length?" LMSYS Org Blog, 2023.

Questions?

Let’s discuss where co-design can move the practical frontier.

THANK YOU

Decoupled layers, coordinated policy

L1 · PRIORITY ENGINE

scores token importance per decode step · pins essential context (system prompt, attention sinks, recent window) · never evicts guardrail tokens

1

L2 · CACHE FORMAT MANAGER

assigns FP16 to high-priority tokens · INT4 to low-priority · packs cache blocks · tracks GPU / CPU residency table

2

L3 · TRANSFER SCHEDULER

prefetches next-turn KV entries from CPU RAM · uses dual CUDA streams to overlap transfer with active GPU decode · hides PCIe cost

3

Dynamic attention-based priority

Observe

rolling attention + recency

Score

importance × persistence

Protect

sinks + system + recent

Act

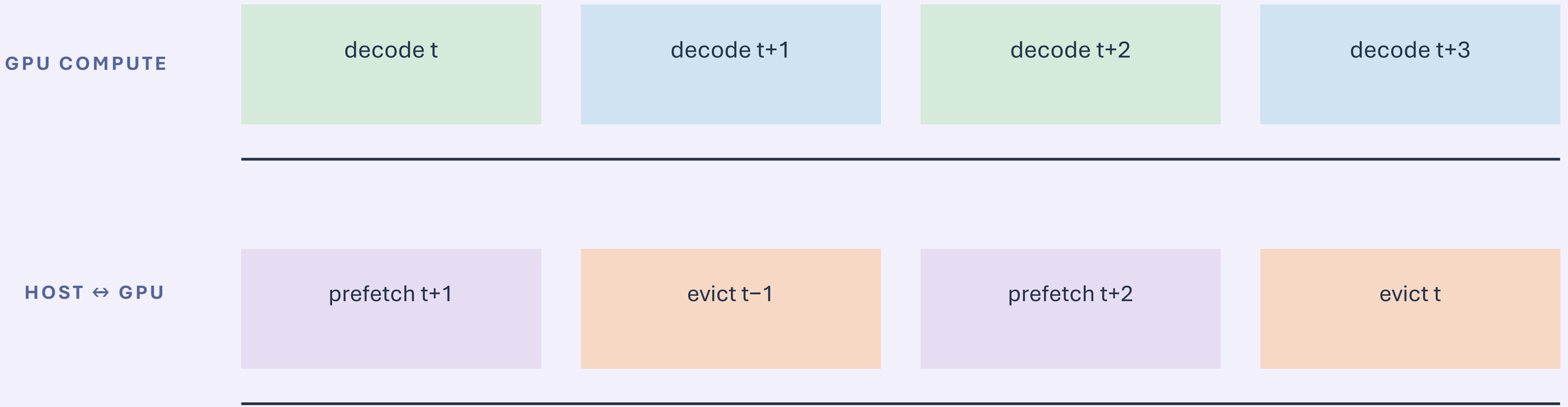
keep · compress · evict

Zero-loss guardrail

NON-NEGOTIABLE

Essential reasoning context is pinned before any adaptive action: system instructions, attention sinks, recent window, and policy-defined anchors.

Hide PCIe latency behind useful work



Dual CUDA streams + pinned host buffers + look-ahead residency scheduling